Conventional Test Design Document

Revision note: initial deliverables were expanded, but this version concentrates on testing the NotebookLM Q&A AI feature per project direction (deliverables 0 and 1 updated accordingly).

This document describes the test design for the **Semantic Similarity Test Automation** project. The document follows the deliverable structure specified for the team-based project.

Section 1 – Introduction

Project: NotebookLM Semantic Similarity Test Automation

Target Application: Google NotebookLM

Description: A Python-based test automation framework specifically designed to validate the semantic accuracy of Google's NotebookLM AI system. **Importantly, the scope is limited to the Q&A feature of NotebookLM rather than the entire AI product; this reflects a late project realization that deliverables should focus on a single feature.** The framework automates testing of NotebookLM's Q&A responses by comparing AI-generated answers against expected content using semantic similarity metrics. It includes a FastAPI service, embedding and similarity utilities, Selenium/Playwright browser automation tests, and helper scripts for test data augmentation and offline scoring.

Team and Task Partition: - *Lead Coordinator*: overview and integration - *Backend Developer*: maintain `semantic/` modules, API - *Test Automation Engineer*: develop browser-based tests and offline scoring scripts - *DevOps/Infrastructure*: virtual environment, Dockerfile, environment variables

Project Schedule: 1. Setup environment and install dependencies (Week 1) 2. Implement semantic utilities and API (Week 2) 3. Develop initial test cases and scoring helpers (Week 3) 4. Extend with augmentation and Selenium/Playwright tests (Week 4) 5. Consolidate documentation and run full test suite (Week 5)

Section 2 – Test Requirements

Scope: - Functional testing of the **NotebookLM Q&A feature only**; other capabilities of NotebookLM (note-taking, summarization, etc.) are out of scope. - Semantic comparison of AI responses against expected answers and source passages - Validation of citation linking (verifying NotebookLM cites correct sources) - Testing NotebookLM's handling of various question types (simple

facts, multi-fact, list answers, exact definitions, edge cases) - Consistency checks across multiple question variations

NotebookLM AI Function Requirements: - **Answer Generation:** NotebookLM must generate contextually relevant answers from uploaded source documents - **Citation Accuracy:** NotebookLM must correctly identify and link source passages that support each answer - **Semantic Correctness:** AI answers must match expected content semantically (not just textually) - **Consistency:** Similar/paraphrased questions should yield semantically similar answers - **Edge Case Handling:** NotebookLM should properly handle questions about unavailable content, ambiguous queries, and complex multi-part questions

High-level Scenario Diagrams:

```
[Upload Source Document] -> [Ask Question via NotebookLM UI]
          ↓
[Capture HTML Response & Citation Links] -> [Extract AI Answer & Source Passage]
          ↓
[Compute Semantic Embeddings] -> [Calculate Cosine Similarity]
          ↓
[Compare Against Threshold] -> [Pass/Fail Assertion]
          ↓
[Log Results to CSV with Timestamp]
```

Analysis Model: - **Question Type Classification:** Simple facts, list answers, multi-fact, definitions, non-available content, consistency checks - **Citation Validation:** Verify that clicked citations show the actual source passage from uploaded documents - **Semantic Scoring:** Measure similarity between AI answer and expected passage using sentence-transformers embeddings - **Threshold-based Decision:** Pass if semantic similarity configured threshold (default 0.65); fail otherwise

---

Section 3 – Selected Test Models and Methods

Selected Test Models: - Black-box testing for NotebookLM UI interactions and API behavior - Regression testing for consistency of semantic similarity scores across test runs - End-to-end system tests via browser automation (Selenium/Playwright) simulating real user workflows

Test Methods: - **Unit Tests (pytest):** Test embedding generation, cosine similarity calculations, and helper functions in isolation - **Integration Tests:** Verify the complete pipeline from capturing HTML artifacts to computing offline semantic scores - **System Tests (Browser Automation):** Use Selenium/Playwright to automate NotebookLM interactions: - Navigate to NotebookLM, open a notebook - Submit test queries - Extract AI responses and citation links - Click citations to verify source passages - Compare responses

semantically against expected content - **Regression Tests:** Re-run scoring on previous test artifacts to detect model/threshold regressions

Coverage Criteria: - **NotebookLM Question Types:** All major question categories (simple facts, list answers, multi-fact, definitions, edge cases) - **API Routes:** All embedding and similarity endpoints exercised - **Browser Selectors:** All UI elements required for NotebookLM interaction covered - **Code Coverage:** 80% statement coverage in core `semantic/` modules - **Threshold Validation:** Test both pass (score threshold) and fail (score < threshold) scenarios

---

Section 4 – Test Case Design with Test Data

Below are representative test cases; more cases follow similar templates.

Example Test Cases

TC ID	Question Type	Test Case	Preconditions	Steps	Expected Result	Notes
TC-01	Simple Fact	Verify exact definition answer	NotebookLM open with "City Data.md" notebook	1. Ask "What is the capital of France?" 2. Capture response 3. Extract expected/actual 4. Compute similarity	Semantic score 0.65; citation links to "City Data.md"	Baseline test; verifies basic Q&A accuracy

TC ID	Question Type	Test Case	Preconditions	Steps	Expected Result	Notes
TC-02	List Answer	Verify list comprehensiveness	Source contains list of 3 items	1. Ask "List three primary colors" 2. Extract answer 3. Score against expected answer	All 3 colors mentioned; score 0.65	Tests ability to generate complete lists
TC-03	Multi-fact	Verify multi-part answer	Query requires multiple facts	1. Ask complex question with 2+ parts 2. Capture full response 3. Score each part independently	Each semantic score 0.65	More challenging; requires integrated understanding

TC ID	Question Type	Test Case	Preconditions	Steps	Expected Result	Notes
TC-04	Exact Definition	Verify definition precision	Source has explicit definition	1. Ask "What is X?" 2. Verify answer matches definition 3. Score response	Exact match or high semantic similarity (0.70)	Strictest test; definition must align closely
TC-05	Non-available Content	Verify handling of missing info	Question about content not in notebook	1. Ask about unavailable topic 2. Capture NotebookLM's refusal/uncertainty response 3. Log behavior	NotebookLM indicates unavailable content (qualitative check)	Edge case; no scoring, behavioral verification only

TC ID	Question Type	Test Case	Preconditions	Steps	Expected Result	Notes
TC-06	Consistency Check	Verify paraphrased questions yield similar answers	Same question asked 3 ways	1. Ask original question 2. Ask paraphrase #1 3. Ask paraphrase #2 4. Compare all 3 responses	All 3 answers semantic similarity 0.65 to each other	Tests robustness and consistency

TC ID	Question Type	Test Case	Preconditions	Steps	Expected Result	Notes
TC-07	Citation Verification	Verify clicked citation shows correct source	Response contains citation link	1. Capture response 2. Click citation button 3. Extract source passage 4. Verify passage matches expected content	Source panel displays correct text; no broken links	Critical for trustworthiness
TC-08	Multi-question Consistency	Verify notebook state maintained across queries	Multiple sequential questions	1. Ask Q1 2. Ask Q2 3. Ask Q1 again 4. Compare Q1 responses	Both Q1 responses semantically similar (0.65)	Tests session consistency

Test Data: - **Source Document:** "City Data.md" and "Nations and Num-

bers" notebooks uploaded to NotebookLM - **Query Strings:** Hardcoded test queries for each question type (e.g., "What is the capital of France?") - **Expected Answers:** Pre-defined reference texts for comparison - **Artifacts:** Directory holding captured HTML, extracted text, and semantic scores

---

Section 5 – Test Result Analysis and Bug Summary

Test Result Analysis: - pytest generates pass/fail counts for unit and integration tests - Offline scoring CSV (`semantic_results.csv`) aggregates semantic similarity scores for each NotebookLM query: - Timestamp (ISO 8601), test name, expected text, actual response, semantic score - Metrics tracked: - **Manual Test Effort:** ~900 lines of test code; Selenium test maintenance is ongoing due to UI changes - **Complexity:** High (asynchronous browser interactions, dynamic selectors, network variability) - **Citation Extraction Difficulty:** Medium (requires HTML parsing and BeautifulSoup) - **Semantic Scoring Reliability:** Moderate (depends on sentence-transformers model consistency)

Bug Summary & Common Issues: 1. **UI Selector Breakage** (Severity: High) - Issue: NotebookLM UI changes break Selenium XPath selectors - Fix: Regularly update selectors in test files (TC-01, TC-07)

2. **Embedding Model Changes** (Severity: Medium)
   - Issue: Updating sentence-transformers model shifts similarity scores, causing threshold mismatches
   - Fix: Baseline and re-score existing tests when model updates
3. **Network Instability** (Severity: Medium)
   - Issue: Flaky citations or slow response loading during browser tests
   - Fix: Increase WebDriver wait timeouts; add retry logic
4. **Missing Corpus Data** (Severity: Low)
   - Issue: API search endpoint fails if corpus is not provided
   - Fix: Ensure request includes `corpus` list or pre-populate default
5. **Threshold False Positives** (Severity: Medium)
   - Issue: Legitimate answers score below threshold (e.g., 0.63) due to phrasings
   - Fix: Review and adjust threshold based on real-world test results

Coverage Summary: - **Functional Coverage:** All 7 question types covered (simple, list, multi-fact, definition, non-available, consistency, multi-question) - **Citation Coverage:** Citation verification and source passage extraction tested - **API Coverage:** Embedding, similarity, and search endpoints exercised - **Browser Coverage:** Primary NotebookLM notebook tested; additional notebooks can be added - **Threshold Validation:** Both passing and failing scenarios included

Recommended Improvements: 1. Implement visual regression testing for NotebookLM UI consistency 2. Add performance benchmarks (response time

per query) 3. Expand test data with more diverse question types 4. Automate threshold calibration based on test results distribution 5. Integrate continuous monitoring of embedding model updates

---

This document satisfies the deliverable requirements for the Conventional Test Design Document.